HTTP Haven

The goal of the exercise isn’t to complete tasks but to build understanding.

DO NOT RUSH!

The use of the internet or external resources is not allowed.

https://rebrand.ly/lettherebecode

1- Repo to submit **when you are done:** http-haven

2- Submit each exercise as a separate file in the http-haven directory.

3- Collaboration with peers is permitted. Your fellows are the ONLY source of truth.

Tests

Terminal Test Script (`test_endpoints.sh`)

This shell script acts as an automated collection of `curl` requests. You can run this directly from their terminal to verify if their 7 endpoints match your performance requirements.

Create a file named `test_endpoints.sh`, paste the code below, and make it executable using `chmod +x test_endpoints.sh`.

Use `./test_endpoints.sh` to run the tests.

```
#!/bin/bash

# Configuration
SERVER_URL="http://localhost:8080"
GREEN='\033[0;32m'
RED='\033[0;31m'
BLUE='\033[0;34m'
NC='\033[0;0m' # No Color

echo -e "${BLUE}=== Starting Go HTTP Exercise Verification Script ===${NC}\n"

# Exercise 1: Basic Ping-Pong Server
echo -e "${BLUE}[Exercise 1: /ping]${NC}"
RESPONSE=$(curl -s "$SERVER_URL/ping")
if [ "$RESPONSE" == "pong" ]; then
    echo -e "${GREEN}✔ PASS: Got 'pong'${NC}"
else
    echo -e "${RED}✘ FAIL: Expected 'pong', got '$RESPONSE'${NC}"
fi
echo ""

# Exercise 2: Query Parameters & Path Validation
echo -e "${BLUE}[Exercise 2: /hello]${NC}"
# Test with name
RESP_NAME=$(curl -s "$SERVER_URL/hello?name=Alice")
if [[ "$RESP_NAME" == *"Hello, Alice!"* ]]; then
    echo -e "${GREEN}✔ PASS: Query param parsed successfully ('Hello, Alice!')${NC}"
else
    echo -e "${RED}✘ FAIL: Expected 'Hello, Alice!', got '$RESP_NAME'${NC}"
fi

# Test default guest
```

```
RESP_GUEST=$(curl -s "$SERVER_URL/hello")
if [[ "$RESP_GUEST" == *"Hello, Guest!"* ]]; then
    echo -e "${GREEN}✔ PASS: Default fallback working ('Hello, Guest!')${NC}"
else
    echo -e "${RED}✘ FAIL: Expected fallback, got '$RESP_GUEST'${NC}"
fi

# Test invalid method validation
STATUS_CODE=$(curl -s -o /dev/null -w "%{http_code}" -X POST "$SERVER_URL/hello")
if [ "$STATUS_CODE" == "405" ]; then
    echo -e "${GREEN}✔ PASS: Blocked POST request with Status 405 Method Not
Allowed${NC}"
else
    echo -e "${RED}✘ FAIL: POST request gave status $STATUS_CODE instead of 405${NC}"
fi
echo ""

# Exercise 3: Text Counter
echo -e "${BLUE}[Exercise 3: /count]${NC}"
# Test GET
RESP_GET=$(curl -s "$SERVER_URL/count")
if [[ "$RESP_GET" == *"Send a POST request"* ]]; then
    echo -e "${GREEN}✔ PASS: GET request displays instruction text${NC}"
else
    echo -e "${RED}✘ FAIL: Unexpected GET response '$RESP_GET'${NC}"
fi

# Test POST
RESP_POST=$(curl -s -X POST -d "Golang" "$SERVER_URL/count")
if [[ "$RESP_POST" == *"6"* ]]; then
    echo -e "${GREEN}✔ PASS: POST request calculated length correctly ('Golang' = 6)${NC}"
else
    echo -e "${RED}✘ FAIL: Expected length 6, got '$RESP_POST'${NC}"
fi
echo ""

# Exercise 4: Basic Math API
echo -e "${BLUE}[Exercise 4: /calculate]${NC}"
# Test valid math
RESP_MATH=$(curl -s "$SERVER_URL/calculate?op=add&a=12&b=8")
if [[ "$RESP_MATH" == *"20"* ]]; then
    echo -e "${GREEN}✔ PASS: 12 + 8 = 20 handled successfully${NC}"
else
    echo -e "${RED}✘ FAIL: Expected 20, got '$RESP_MATH'${NC}"
```

```
fi

# Test invalid input validation
STATUS_MATH=$(curl -s -o /dev/null -w "%{http_code}"
"$SERVER_URL/calculate?op=multiply&a=abc&b=5")
if [ "$STATUS_MATH" == "400" ]; then
    echo -e "${GREEN}✔ PASS: Rejected non-integer values with Status 400 Bad
Request${NC}"
else
    echo -e "${RED}✘ FAIL: Expected status 400 for bad parameters, got
$STATUS_MATH${NC}"
fi
echo ""

# Exercise 5: User-Agent Echo
echo -e "${BLUE}[Exercise 5: /agent]${NC}"
RESP_AGENT=$(curl -s -H "User-Agent: CustomTester/1.0" "$SERVER_URL/agent")
if [[ "$RESP_AGENT" == *"CustomTester/1.0"* ]]; then
    echo -e "${GREEN}✔ PASS: Header value extracted and echoed back${NC}"
else
    echo -e "${RED}✘ FAIL: Expected agent info to be visible, got '$RESP_AGENT'${NC}"
fi
echo ""

# Exercise 6: Secure Dashboard
echo -e "${BLUE}[Exercise 6: /dashboard]${NC}"
# Test unauthorized access
STATUS_DASH_BAD=$(curl -s -o /dev/null -w "%{http_code}" "$SERVER_URL/dashboard")
if [ "$STATUS_DASH_BAD" == "401" ]; then
    echo -e "${GREEN}✔ PASS: Missing API key blocked with Status 401 Unauthorized${NC}"
else
    echo -e "${RED}✘ FAIL: Expected status 401 for unauthorized traffic, got
$STATUS_DASH_BAD${NC}"
fi

# Test authorized access
RESP_DASH_GOOD=$(curl -s -H "X-API-Key: secret123" "$SERVER_URL/dashboard")
if [[ "$RESP_DASH_GOOD" == *"Welcome"* ]]; then
    echo -e "${GREEN}✔ PASS: Access granted with correct token header${NC}"
else
    echo -e "${RED}✘ FAIL: Correct token rejected. Response: '$RESP_DASH_GOOD'${NC}"
fi
echo ""
```

```
# Exercise 7: Simple Redirector
echo -e "${BLUE}[Exercise 7: /legacy -> /v2]${NC}"
# Test redirect status
REDIRECT_STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$SERVER_URL/legacy")
if [ "$REDIRECT_STATUS" == "301" ]; then
    echo -e "${GREEN}✔ PASS: Route /legacy issues a 301 Permanent Redirect${NC}"
else
    echo -e "${RED}✘ FAIL: Expected redirect status 301, got $REDIRECT_STATUS${NC}"
fi

# Test following the location redirect
RESP_REDIRECT=$(curl -s -L "$SERVER_URL/legacy")
if [[ "$RESP_REDIRECT" == *"version 2"* ]]; then
    echo -e "${GREEN}✔ PASS: Followed redirect pipeline to /v2 successfully${NC}"
else
    echo -e "${RED}✘ FAIL: Target redirection payload path failed. Got:
'$RESP_REDIRECT'${NC}"
fi

echo -e "\n${BLUE}=== Testing Complete ===${NC}"
```

Exercise 1

Exercise 1: Basic Ping-Pong Server

Goal: Build a minimal web server that listens on port `8080` and responds with "pong" when a user visits the `/ping` route.

Tasks:

- Create a route handler for `/ping` using `http.HandleFunc`.
- Use `w.Write()` or `fmt.Fprint()` to send a plain text response.
- Start the server on port `:8080` using `http.ListenAndServe`.

Exercise 2

Exercise 2: Query Parameters & Path Validation

Goal: Create a `/hello` endpoint that reads a `name` query parameter (e.g., `/hello?name=Alice`) and responds with "Hello, Alice!". If the parameter is missing, default to "Hello, Guest!".

Tasks:

- Extract query parameters using `r.URL.Query().Get("name")`.
- Reject any HTTP method that is not `GET` by returning an `http.StatusMethodNotAllowed` status code.

Exercise 3

Exercise 3: Text Counter (URL Variables & Methods)

Goal: Build a server with a `/count` route. If a user sends a `GET` request, return the text "Send a POST request with text to count words". If they send a `POST` request, read the text body and return the number of characters.

Key Tasks:

- Differentiate between `GET` and `POST` methods using `r.Method`.
- Read the entire request body using `io.ReadAll(r.Body)`.
- Return the character length as a string.

Exercise 4

Exercise 4: Basic Math API (Multiple Query Parameters)

Goal: Create a `/calculate` route that accepts three query parameters: `op` (operation), `a`, and `b`. For example, `/calculate?op=add&a=10&b=5` should respond with `Result: 15`.

Key Tasks:

- Parse string query variables using `r.URL.Query().Get()`.
- Convert string inputs to integers using `strconv.Atoi()`.
- Support `add`, `subtract`, and `multiply`. Return an HTTP `400 Bad Request` if the operation is unknown or parsing fails.

Exercise 5

Exercise 5: User-Agent Echo (Reading Headers)

Goal: Create an `/agent` route that reads the incoming browser or client header details and echoes it back in plain text: `"You are visiting us using: [User-Agent Info]"`.

Key Tasks:

- Inspect request headers using `r.Header.Get("User-Agent")`.
- Handle instances where the header might be missing or empty.

Exercise 6

Exercise 5: Secure Dashboard (Simple Authorization Headers)

Goal: Create a `/dashboard` route that acts as a protected page. If the client does not provide a specific API key in their headers, block them.

Key Tasks:

- Read custom headers using `r.Header.Get("X-API-Key")`.
- Match it against a hardcoded value (e.g., `secret123`).
- Use `http.StatusUnauthorized` (401) to reject bad keys.

Exercise 7

Exercise 6: Simple Redirector (Status Codes)

Goal: Create a `/legacy` route. Whenever a user hits this endpoint, permanently redirect them to a new route `/v2` with a friendly "Welcome to version 2" message.

Key Tasks:

- Redirect traffic using the `http.Redirect` helper function.
- Use the proper status code for a permanent move (`http.StatusMovedPermanently`).

Cheat Sheet

Go HTTP Server Cheat Sheet

Core Standard Library Functions

1. Routing & Server Management (`net/http`)

- **`http.HandleFunc(pattern string, handler func(ResponseWriter, *Request))`**
  Registers a handler function for a specific URL path pattern.
- **`http.ListenAndServe(addr string, handler Handler) error`**
  Starts an HTTP server on the specified address (e.g., `":8080"`). Passing `nil` uses the default system router (`DefaultServeMux`).
- **`http.Error(w ResponseWriter, error string, code int)`**
  A helper that sends a specific error message string and numeric HTTP status code back to the client, automatically ending the request lifecycle safely.
- **`http.Redirect(w ResponseWriter, r *Request, url string, code int)`**
  Sends an HTTP redirect status code (like `301` or `302`) forcing the client's browser to jump to a new target URL path.

2. Reading Inputs & Formatting (`io`, `strconv`, `fmt`)

- **`io.ReadAll(r io.Reader) ([]byte, error)`**
  Reads all remaining data from an input stream (like `r.Body`) until it hits the end of the file/stream (`EOF`). Returns a byte slice.
- **`strconv.Atoi(s string) (int, error)`**
  Stands for "ASCII to Integer". Converts a text string into a native numeric `int`. Returns an error if the text contains non-numeric characters.
- **`fmt.Fprintf(w io.Writer, format string, a ...any)`**
  Formats data according to a template string and writes the output directly into an open network socket stream or file (`w`).

Request Context Breakdown (`*http.Request`)

When a client hits your server, all incoming information is bundled inside the pointer to the `http.Request` struct (usually named `r`):

Struct Field / Method	Purpose / Explanation	Example Usage
`r.Method`	A string representing the incoming HTTP type. Always use standard library constants for comparisons.	`if r.Method == http.MethodPost`
`r.Body`	An open stream containing data uploaded via POST/PUT requests. **Always** close it after reading to prevent memory leaks.	`defer r.Body.Close()`
`r.URL.Query()`	Parses the raw URL query string (everything after the `?`) and returns a map-like structure (`Values`).	`queryMap := r.URL.Query()`
`r.URL.Query().Get(key)`	Fetches the value of a specific query parameter. Returns an empty string `""` if the key does not exist.	`name := r.URL.Query().Get("name")`
`r.Header.Get(key)`	Fetches the value of a specific HTTP request header (case-insensitive). Returns `""` if missing.	`token := r.Header.Get("X-API-Key")`

Response Management (`http.ResponseWriter`)

The `http.ResponseWriter` interface (usually named `w`) is your pipeline to send data back to the client. Keep this execution order in mind:

1. **Modify Headers First:** Call `w.Header().Set("Key", "Value")` before anything else if changing content types or metadata.
2. **Write Status Code Second:** Call `w.WriteHeader(http.StatusCreated)` if returning something other than a standard `200 OK`.
3. **Write Body Content Last:** Call `w.Write([]byte)` or `fmt.Fprintf(w)` to push actual visual content to the user. Writing to the body locks your headers and status code automatically.

HTTP Resurgence

The goal of the exercise isn’t to complete tasks but to build understanding.

DO NOT RUSH!

The use of the internet or external resources is not allowed.

https://rebrand.ly/lettherebecode

1- Repo to submit **when you are done:** http-resurgence

2- Submit each exercise as a separate file in the http-resurgence directory.

3- Collaboration with peers is permitted. Your fellows are the ONLY source of truth.

v2 Tests

Terminal Test Script (`test_endpoints.sh`)

This shell script acts as an automated collection of `curl` requests. You can run this directly from their terminal to verify if their 7 endpoints match your performance requirements.

Create a file named `test_endpoints.sh`, paste the code below, and make it executable using `chmod +x test_endpoints.sh`.

Use `./test_endpoints.sh` to run the tests.

```
#!/bin/bash

SERVER_URL="http://localhost:8080"

GREEN='\033[0;32m'

RED='\033[0;31m'

BLUE='\033[0;34m'

NC='\033[0;0m'


echo -e "${BLUE}=== HTTP Resurgence Verification ===${NC}\n"


# Exercise 1: /method-inspector

echo -e "${BLUE}[Exercise 1: /method-inspector]${NC}"

R1=$(curl -s -X GET "$SERVER_URL/method-inspector")

if [[ "$R1" == *"GET"* ]]; then echo -e "${GREEN}✔ PASS: GET detected${NC}"; else echo -e "${RED}✘ FAIL: got '$R1'${NC}"; fi

R1P=$(curl -s -X POST "$SERVER_URL/method-inspector")

if [[ "$R1P" == *"POST"* ]]; then echo -e "${GREEN}✔ PASS: POST detected${NC}"; else echo -e "${RED}✘ FAIL: got '$R1P'${NC}"; fi
```

```
# Exercise 2: /echo

echo -e "\n${BLUE}[Exercise 2: /echo]${NC}"

R2=$(curl -s -X POST -d "Hello Go" "$SERVER_URL/echo")

if [[ "$R2" == *"Hello Go"* ]]; then echo -e "${GREEN}✔ PASS: body echoed${NC}"; else echo -e "${RED}✘ FAIL: got '$R2'${NC}"; fi

R2G=$(curl -s -o /dev/null -w "%{http_code}" -X GET "$SERVER_URL/echo")

if [ "$R2G" == "405" ]; then echo -e "${GREEN}✔ PASS: GET blocked with 405${NC}"; else echo -e "${RED}✘ FAIL: expected 405 got $R2G${NC}"; fi


# Exercise 3: /headers

echo -e "\n${BLUE}[Exercise 3: /headers]${NC}"

R3=$(curl -s -H "X-Custom-Token: abc123" "$SERVER_URL/headers")

if [[ "$R3" == *"abc123"* ]]; then echo -e "${GREEN}✔ PASS: header echoed${NC}"; else echo -e "${RED}✘ FAIL: got '$R3'${NC}"; fi

R3E=$(curl -s "$SERVER_URL/headers")

if [[ "$R3E" == *"missing"* || "$R3E" == *"Missing"* ]]; then echo -e "${GREEN}✔ PASS: missing header handled${NC}"; else echo -e "${RED}✘ FAIL: got '$R3E'${NC}"; fi


# Exercise 4: /form

echo -e "\n${BLUE}[Exercise 4: /form]${NC}"

R4=$(curl -s -X POST -d "username=Ada&language=Go" "$SERVER_URL/form")

if [[ "$R4" == *"Ada"* && "$R4" == *"Go"* ]]; then echo -e "${GREEN}✔ PASS: form parsed${NC}"; else echo -e "${RED}✘ FAIL: got '$R4'${NC}"; fi

R4E=$(curl -s -o /dev/null -w "%{http_code}" -X POST -d "username=&language=Go" "$SERVER_URL/form")

if [ "$R4E" == "400" ]; then echo -e "${GREEN}✔ PASS: empty field returns 400${NC}"; else echo -e "${RED}✘ FAIL: expected 400 got $R4E${NC}"; fi
```

```
# Exercise 5: /status

echo -e "\n${BLUE}[Exercise 5: /status]${NC}"

R5=$(curl -s -o /dev/null -w "%{http_code}" "$SERVER_URL/status?code=404")

if [ "$R5" == "404" ]; then echo -e "${GREEN}✔ PASS: 404 returned${NC}"; else echo -e
"${RED}✘ FAIL: expected 404 got $R5${NC}"; fi

R5B=$(curl -s -o /dev/null -w "%{http_code}" "$SERVER_URL/status?code=banana")

if [ "$R5B" == "400" ]; then echo -e "${GREEN}✔ PASS: bad code returns 400${NC}"; else echo
-e "${RED}✘ FAIL: expected 400 got $R5B${NC}"; fi



# Exercise 6: /api/v1/greet (ServeMux subtree)

echo -e "\n${BLUE}[Exercise 6: /api/v1/greet]${NC}"

R6=$(curl -s "$SERVER_URL/api/v1/greet?name=Zion")

if [[ "$R6" == *"Zion"* ]]; then echo -e "${GREEN}✔ PASS: subtree route greet works${NC}";
else echo -e "${RED}✘ FAIL: got '$R6'${NC}"; fi

R6P=$(curl -s "$SERVER_URL/api/v1/ping")

if [[ "$R6P" == *"pong"* ]]; then echo -e "${GREEN}✔ PASS: subtree route ping works${NC}";
else echo -e "${RED}✘ FAIL: got '$R6P'${NC}"; fi



# Exercise 7: /render (template)

echo -e "\n${BLUE}[Exercise 7: /render]${NC}"

R7=$(curl -s "$SERVER_URL/render?title=SENTINEL&body=Online")

if [[ "$R7" == *"SENTINEL"* && "$R7" == *"Online"* ]]; then echo -e "${GREEN}✔ PASS:
template rendered${NC}"; else echo -e "${RED}✘ FAIL: got '$R7'${NC}"; fi

R7E=$(curl -s -o /dev/null -w "%{http_code}" "$SERVER_URL/render")

if [ "$R7E" == "400" ]; then echo -e "${GREEN}✔ PASS: missing params returns 400${NC}";
else echo -e "${RED}✘ FAIL: expected 400 got $R7E${NC}"; fi
```

```
echo -e "\n${BLUE}=== Verification Complete ===${NC}"
```

v2 Exercise 1

Exercise 1: The Method Inspector

Goal

Build a /method-inspector endpoint that reads the HTTP method of every incoming request and echoes it back in a descriptive sentence. No method should be rejected — this handler accepts everything and reports what it sees.

Key Tasks

- Register a /method-inspector handler using http.HandleFunc.
- Read the request method using r.Method.
- Respond with a plain text message that includes the method name.
  - GET request → "You made a GET request."
  - POST request → "You made a POST request."
  - Any other method → "You made a [METHOD] request."
- Do not hardcode each method with its own if/else branch — use the value of r.Method directly in your response string.

Why this matters —

In ascii-art-web your POST /ascii-art handler must distinguish incoming methods before doing any work. This exercise builds the muscle of reading r.Method cleanly and using it — not just checking it.

v2 Exercise 2

Exercise 2: The Echo Chamber

Goal

Create an /echo endpoint that only accepts POST requests. When a client sends a POST with a body, read the entire body and send it straight back. The response must be exactly what was sent — nothing added, nothing removed.

Key Tasks

- Reject any non-POST request with http.StatusMethodNotAllowed (405).
- Read the full request body using io.ReadAll(r.Body).
- Always defer r.Body.Close() immediately after checking for an error — not at the end of the function.
- If the body is completely empty (zero bytes), return a 400 Bad Request with the message "body cannot be empty".
- Write the body content back to w exactly as received.

Think about —

What does io.ReadAll return if the request has no body at all?

Is len(body) == 0 the same as body == nil? Try both and see.

What happens to r.Body if you read it twice without closing it?

Stretch — do this after the core task works

- Add a response header: Content-Type: text/plain before writing the body back.
  - Use w.Header().Set("Content-Type", "text/plain") — and call it before w.Write().
  - What happens if you call w.Header().Set() after w.Write()? Try it and explain what you observe.

v2 Exercise 3

Exercise 3: Header Detective

Goal

Create a /headers endpoint that inspects two specific request headers: X-Custom-Token and Content-Type. The handler reads both, reports what it found, and enforces a rule about one of them.

Key Tasks

- Read X-Custom-Token using r.Header.Get("X-Custom-Token").
- If X-Custom-Token is missing or empty — respond with 400 Bad Request and the message: "X-Custom-Token header is missing".
- If X-Custom-Token is present — respond with a message that includes its value. Example: "Token received: abc123".
- Also read Content-Type and append it to the response. If it is missing, append "Content-Type not provided".
- The full response for a valid request must look like this:
  - Token received: abc123
  - Content-Type: application/json

Why this matters —

ascii-art-web reads r.Header indirectly through template and form handling.
Understanding how headers work — and what happens when they are absent —
prepares you for writing handlers that behave correctly under any input.

Stretch — do this after the core task works

- What does r.Header.Get() return for a header key that was never sent? Write a one-sentence answer in a comment at the top of your file.
- Is r.Header.Get("x-custom-token") the same as r.Header.Get("X-Custom-Token")? Find out.

v2 Exercise 4

Exercise 4: Form Decoder

Goal

Build a /form endpoint that accepts a POST request with a URL-encoded form body containing two fields: username and language. Parse both, validate them, and return a formatted confirmation. This is the closest exercise in this set to what ascii-art-web actually does.

Key Tasks

- Reject non-POST requests with 405.
- Call r.ParseForm() to parse the incoming form body — do not skip this step.
- Read username and language using r.FormValue().
- If either field is empty — return 400 Bad Request with a message identifying which field is missing.
  - Missing username → "username is required"
  - Missing language → "language is required"
- If both are present — respond with: "Hello [username], you are coding in [language]!"

Think about —

What is the difference between r.ParseForm() + r.Form.Get() and just r.FormValue()?
r.FormValue() calls ParseForm internally — but calling ParseForm explicitly first
gives you control over error handling. Try it both ways.

Stretch — do this after the core task works

- Handle the case where the request Content-Type is not application/x-www-form-urlencoded. Return a 415 Unsupported Media Type with a clear message.
  - Read Content-Type from r.Header.Get() and check it before parsing.
  - Test it: curl -X POST -H "Content-Type: text/plain" -d "username=Ada" http://localhost:8080/form

v2 Exercise 5

Exercise 5: Status Code Factory

Goal

Build a /status endpoint that accepts a code query parameter containing any HTTP status code number. The server must respond using that exact status code. This forces you to think about how status codes are set — and when they cannot be changed.

Key Tasks

- Read the code query parameter using r.URL.Query().Get("code").
- If code is missing or empty — return 400 with the message: "code parameter is required".
- Convert code to an integer using strconv.Atoi(). If conversion fails — return 400 with: "code must be a valid integer".
- If the integer is not between 100 and 599 — return 400 with: "code must be a valid HTTP status code (100–599)".
- If the code is valid — respond with that exact status code using w.WriteHeader(code) and a body message: "Responding with status [code]".

Critical rule —

You must call w.WriteHeader(code) BEFORE writing anything to w with w.Write() or fmt.Fprintf(). If you write the body first, Go automatically sends a 200 header and you cannot change it afterwards. The order is: WriteHeader → then Write. Test this deliberately: call w.Write() first, then w.WriteHeader(404). What does curl -v show you? Write your observation in a comment in your file.

Stretch — do this after the core task works

- After calling w.WriteHeader(), append a descriptive name to the body message.
  - ?code=404 → "Responding with status 404 Not Found"
  - Use http.StatusText(code) to get the official status name.

v2 Exercise 6

Exercise 6: The API Subtree

Goal

Build a mini API under the /api/v1/ path prefix using a separate http.ServeMux — not the default mux. This mux handles only /api/v1/ routes. Register two handlers inside it: /api/v1/ping and /api/v1/greet. Mount the whole submux onto the main server at /api/.

What a ServeMux subtree is

Go's http.ServeMux uses a simple rule: a pattern that ends in / matches any path that starts with it. This makes it possible to create a sub-router — a separate ServeMux that handles a subtree of routes — and mount it onto the main mux with http.StripPrefix. The main mux passes all /api/ requests to the submux, which handles them as if the /api prefix did not exist.

```
// The pattern for a subtree — note the trailing slash
mainMux.Handle("/api/", http.StripPrefix("/api", apiMux))

// Inside apiMux, routes are registered WITHOUT /api
apiMux.HandleFunc("/v1/ping", pingHandler)
apiMux.HandleFunc("/v1/greet", greetHandler)

// A request to /api/v1/ping:
// mainMux strips "/api" → apiMux sees "/v1/ping" → routes to pingHandler
```

Key Tasks

- Create a new mux: apiMux := http.NewServeMux()
- Register /v1/ping on apiMux — responds with "pong" in plain text.
- Register /v1/greet on apiMux — reads a name query parameter and responds with "Greetings, [name]!" or "Greetings, Stranger!" if name is missing.
- Mount apiMux onto the main mux:
  - mainMux := http.NewServeMux()
  - mainMux.Handle("/api/", http.StripPrefix("/api", apiMux))
- Start the server using http.ListenAndServe(":8080", mainMux) — not nil.
- Visiting /api/v1/ping must return "pong". Visiting /api/v1/greet?name=Zion must return "Greetings, Zion!".

Why this matters —

ascii-art-web mounts handlers at / and /ascii-art. A real application groups related routes under a prefix — /api/, /admin/, /v2/. ServeMux subtrees are Go's standard way to do this without an external router. Understanding StripPrefix is the foundation of any multi-route Go server.

Think about — write your answers in comments at the top of your file

- What happens if you use mainMux.Handle("/api", ...) without the trailing slash? Try it.
- What does http.StripPrefix do — what would apiMux receive if you did NOT use StripPrefix?
- What does it mean that http.ListenAndServe takes a Handler interface — and how does a *http.ServeMux satisfy that interface?

v2 Exercise 7

Exercise 7: Template Renderer

Goal

Build a /render endpoint that accepts two query parameters — title and body — and renders them into an inline HTML template. The template must be defined inside your Go file as a string constant, parsed with html/template, and executed into the ResponseWriter. No external HTML files.

Key Tasks

- Define an HTML template as a raw string constant inside your .go file:

```
const tmplStr = `
<!DOCTYPE html>
<html>
<head><title>{{.Title}}</title></head>
<body>
  <h1>{{.Title}}</h1>
  <p>{{.Body}}</p>
</body>
</html>
`

type PageData struct {
      Title string
      Body  string
}
```

- Parse the template using template.Must(template.New("page").Parse(tmplStr)).
- In the /render handler — read title and body from the query string.
- If either is missing — return 400 with: "title and body are required".
- Execute the template with tmpl.Execute(w, PageData{Title: title, Body: body}).
- If Execute returns an error — return 500 with: "template execution failed".
- Set the Content-Type header to text/html before executing the template.

Critical difference from Exercise 2 —

In Exercise 2 you set Content-Type before writing the body. Here you must also set it before calling tmpl.Execute() — because Execute writes to w directly. Once Execute writes its first byte, headers are locked. Set them first.

What is template.Must() — and when does it panic? Write the answer in a comment.

Stretch — do this after the core task works

- Add a third query parameter: style. If style=bold, wrap {{.Body}} in tags.
  - Hint: you cannot use an if statement in the query param — use template conditionals: {{if eq .Style \"bold\"}}{{.Body}}{{else}}{{.Body}}{{end}}
- Add a PageData field Style string and pass it through from the query param.

v2 Cheat Sheets

Core Functions — net/http

Function / Type	What it does
http.NewServeMux()	Creates a new, independent request multiplexer (router). Pass it to ListenAndServe instead of nil to use it as the main router.
http.StripPrefix(prefix, handler)	Returns a handler that strips the given prefix from the request URL before passing it to the next handler. Used to mount a submux.
http.ListenAndServe(addr, handler)	Starts the server. Pass a *ServeMux or nil (uses DefaultServeMux). Never returns unless the server crashes.
w.Header().Set(key, value)	Sets a response header. Must be called BEFORE w.WriteHeader() or any w.Write() call.
w.WriteHeader(code)	Sends the HTTP status code. Must be called BEFORE w.Write(). Calling it after w.Write() has no effect.
http.StatusText(code)	Returns the official status text for a code. e.g. http.StatusText(404) returns "Not Found".
template.Must(t, err)	Wraps a template parse call. Panics if err is not nil — use only at startup, never inside a handler.
template.New(name).Parse(str)	Parses an HTML template from a string. Returns (*Template, error).
tmpl.Execute(w, data)	Renders the template with data and writes the result to w. Returns an error if rendering fails.

The ServeMux Subtree — Pattern

```
func main() {
    // The main mux — receives all requests
    mainMux := http.NewServeMux()

    // A sub-mux — handles only /api/* routes
    apiMux := http.NewServeMux()
    apiMux.HandleFunc("/v1/ping", pingHandler)
    apiMux.HandleFunc("/v1/greet", greetHandler)

    // Mount apiMux under /api/ — StripPrefix removes "/api"
    // so apiMux sees /v1/ping instead of /api/v1/ping
    mainMux.Handle("/api/", http.StripPrefix("/api", apiMux))

    // Register other top-level routes on mainMux
    mainMux.HandleFunc("/ping", pingHandler)

    http.ListenAndServe(":8080", mainMux)
}
```

Response Header Order — The Rule

Step	Call	Notes
1	w.Header().Set("Content-Type", "text/html")	Set ALL headers here. Order matters — before everything.
2	w.WriteHeader(http.StatusCreated)	Set the status code if it is not 200. Only call once.
3	fmt.Fprintf(w, "...") or tmpl.Execute(w, data)	Write the body last. Writing the body locks headers and status.

Request Fields at a Glance

Field / Method	Purpose	Returns when missing
r.Method	The HTTP method — GET, POST, etc.	Never missing — always set
r.URL.Query().Get(key)	A query parameter value	Empty string ""
r.Header.Get(key)	A request header value	Empty string ""
r.FormValue(key)	A form field from POST body or query string	Empty string ""
io.ReadAll(r.Body)	The raw request body as []byte	Empty slice, nil error
r.Body.Close()	Frees the connection — always defer this	—